

Agentic Sprawl to Enterprise Spool

A first look at AWS Agent Registry

Anil Yanamandra
Principal Engineer - Applied AI
AWS Community Builder

PREVIEW · APRIL 2026

1

DISCOVER

Find approved resources by name or intent.

2

GOVERN

Control what becomes discoverable.

3

REUSE

Make vetted capabilities easier to find.

SPEAKER

Anil Yanamandra

Principal Engineer - Applied AI

AWS Community Builder

Anil is a Principal Engineer focused on Applied AI with more than 18 years of experience building global, consumer-facing applications. He applies generative AI to transform legacy enterprise systems and bridges emerging research with production-ready engineering.

18+ YEARS

Building global consumer applications

APPLIED AI

Modernizing enterprise processes with GenAI

BUILDER MINDSET

Turning research into production systems

The problem changed

The experiment question was capability. The enterprise question is trust and reuse.

Can we build it?

1

Prototype

One team, one workflow

2

Adoption

More teams wrap APIs

3

Fleet

Agents, MCP servers, skills

4

Platform

Discovery becomes shared infrastructure

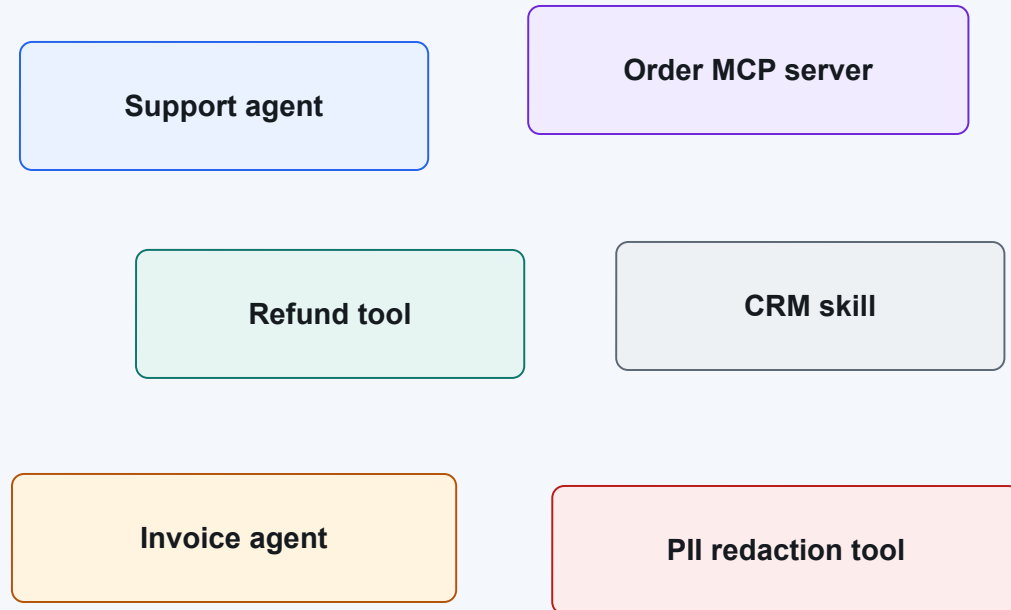
Can we find, trust, and reuse it?

Once resources cross team boundaries, discoverability becomes architecture.

Agentic sprawl is an operating problem

The failure mode is not a lack of ideas. It is a lack of governed visibility.

SCATTERED RESOURCES



No shared owner · status · search path

Visibility

What exists, where is it hosted, and who owns it?

Control

Who can publish, curate, discover, and retire records?

Reuse

Can a team find the capability before rebuilding it?

AWS Agent Registry: first look

A fully managed discovery service for organizing, curating, and discovering agentic resources across an organization.

ORGANIZE

Centralize structured metadata.

CURATE

Approve what enters discovery.

SEARCH

Match exact terms and intent.

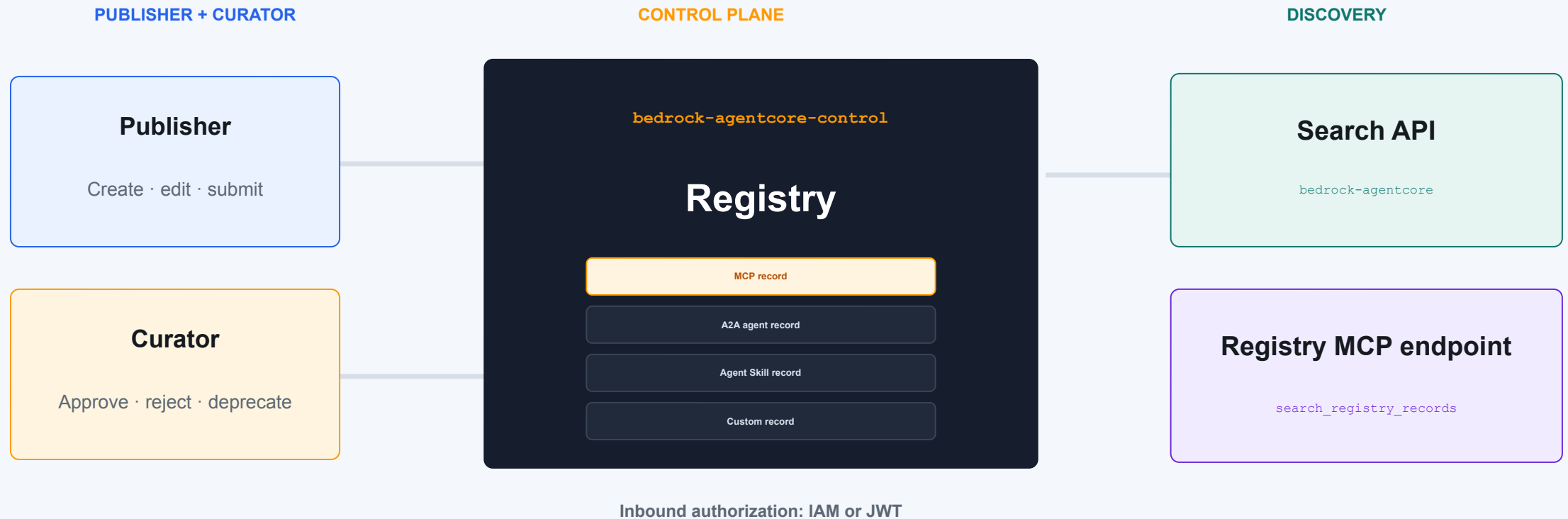
ACCESS

Use APIs, CLI, or MCP clients.

It catalogs metadata. It does not deploy or execute agents.

Control plane in. Discovery surfaces out.

Two core resources - registries and records - sit behind distinct management and discovery interfaces.



Six terms carry the model

Use AWS's vocabulary so architecture, governance, and demo commands stay connected.

Registry	Top-level catalog with authorization, approval settings, and records.	Publisher	Creates, edits, and submits records for approval.
Record	The governed metadata unit representing one agentic resource.	Curator	Approves, rejects, or deprecates records against organizational standards.
Descriptor	Type-specific metadata validated against MCP, A2A, skill, or custom schemas.	Consumer	Searches approved records through the API, CLI, console, or MCP endpoint.

A record is metadata about a resource - not the running resource itself.

The record is the governed discovery unit

Common metadata identifies the record; descriptors explain the underlying protocol and capabilities.

EVERY RECORD

name

description

recordVersion

descriptorType

descriptors

DESCRIPTOR TYPES

MCP

server + tools

A2A agent

agent card

Agent Skills

skill markdown + definition

Custom

organization-defined JSON

DEMO RECORD

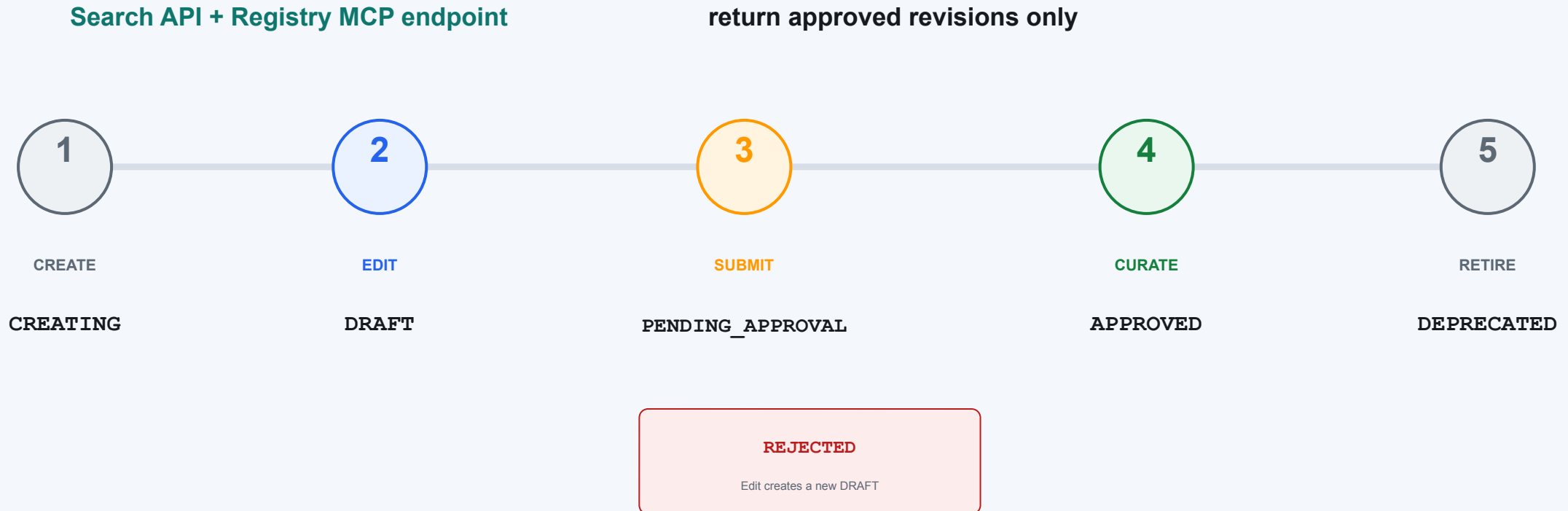
descriptorType: MCP

```
descriptors
  server
    acme/code-review
  tools
    review_code
```

The demo registers an MCP record containing a tool definition.

Approval is the discovery boundary

Lifecycle state determines which record revision consumers can find.



Editing an approved record creates a new DRAFT while the approved revision remains searchable.

Hybrid relevance inside a governance boundary

AWS combines keyword matching with semantic understanding, then filters results through approval state.

SEARCH QUERY

code review security scanning

1-256 characters

1 Hybrid relevance

Keyword matching for every query.
Longer natural-language queries
also use semantic understanding.

2 Approved-only results

DRAFT, PENDING_APPROVAL,
REJECTED, and DEPRECATED
records never appear in search.

3 Operational reality

Indexing is eventually consistent.
Retry with backoff and verify
current status with
GetRegistryRecord.

```
aws bedrock-agentcore search-registry-records --search-query "code review security"
```

The registry is itself an MCP server

Connect a compatible client to the remote endpoint and search governed records without leaving the development environment.

KIRO MCP CONFIG · OAUTH/JWT

```
{
  "mcpServers": {
    "agent-registry": {
      "type": "http",
      "url": "<registry-mcp-endpoint>",
      "headers": {
        "Authorization":
          "Bearer ${ACCESS_TOKEN}"
      }
    }
  }
}
```

ONE MCP TOOL

search_registry_records

searchQuery	required natural-language query
maxResults	1-20, default 10
filter	name, descriptorType, version

IAM-based IDE clients can connect through mcp-proxy-for-aws; OAuth/JWT clients can connect over HTTP.

Register, govern, discover

One MCP server record. Four AWS CLI operations. No runtime invocation.

1REGISTER	2SUBMIT	3APPROVE	4DISCOVER
<code>create-registry-record</code>	<code>submit-registry-record-for-approval</code>	<code>update-registry-record-status</code>	<code>search-registry-records</code>
DRAFT	PENDING_APPROVAL	APPROVED	APPROVED RESULT

Demo record `acme_code_review_mcp` contains **server metadata + review_code tool definition**

Switch to the prepared CLI notebook. Search retries account for eventual consistency.

Three things to remember

- 1 Registry catalogs metadata** It can index resources regardless of where they are built or hosted.
- 2 Lifecycle governs discovery** Approved revisions are the records exposed to search and MCP consumers.
- 3 Discovery belongs in the workflow** Use the CLI, API, console, or IDE before another team rebuilds the capability.

Resources

docs.aws.amazon.com/bedrock-agentcore/latest/devguide/registry.html
github.com/aws-labs/agentcore-samples

Start with one governed slice.

AWS CLI sequence

The exact control-plane and data-plane operations used by the demo.

DEMO CHEAT SHEET

```
# Control plane
aws bedrock-agentcore-control create-registry-record ...
aws bedrock-agentcore-control \
  submit-registry-record-for-approval ...
aws bedrock-agentcore-control \
  update-registry-record-status \
  --status APPROVED --status-reason "..."/>


```
Data plane
aws bedrock-agentcore search-registry-records \
 --search-query "code review security scanning" ...
```


```

Control-plane service: bedrock-agentcore-control | Data-plane service: bedrock-agentcore

Lifecycle edge cases worth knowing

The first-look talk uses the main path; these behaviors matter during implementation.

APPROVED + edit

Creates a new DRAFT revision. The approved revision remains searchable.

PENDING + edit

Creates a new DRAFT and discards the pending revision.

REJECTED + edit

Returns to DRAFT and follows the normal approval path.

Any state + deprecate

Moves to DEPRECATED, a terminal state that is not searchable.

Get/List return the latest revision. Search/MCP return the approved revision.